# Guia de Ambiente: do zero até o MinIO + Spark local

Ao final deste guia, o seu computador terá Java, PySpark e um MinIO (object storage) rodando em Docker com os buckets `landing` e `warehouse`. O Postgres, o pgvector e o Metabase **não** entram agora: vamos montá-los juntos nas próximas etapas, como parte da arquitetura para o banco vetorial.

## Objetivo e escopo

| Etapa | Quando | O que é |
|---|---|---|
| Ambiente Python (venv, PySpark) | Vocês, aqui tem a parte final para configurar. | Passos 1 e 2 |
| Variáveis de ambiente (`.env`) | Fizemos na ultima Aula | Passo 3 |
| MinIO em Docker + buckets | Fizemos na ultima Aula | Passos 4 a 6 |
| Verificação do ambiente | Vocês, aqui tem a parte final para testar. | Passo 7 |
| Postgres + pgvector, Metabase | Construímos juntos, depois | Fora deste guia |

Se a verificação do Passo 7 mostrar `OK` em Java, PySpark e MinIO, vocês estão prontos. Qualquer aviso sobre Postgres é esperado por enquanto.

**Ao longo do guia** aparecem marcações de *qualidade e governança*: decisões que parecem detalhe (guardar senha fora do código, fixar versões, separar zonas de dados) mas que sustentam um pipeline confiável.

## Pré-requisitos

Confira os quatro itens abaixo antes de começar. Para Java e PySpark, siga o *Guia de configuração do PySpark* (Windows e Linux).

| Item | Versão | Como conferir |
|---|---|---|
| Docker Desktop (Windows/macOS) ou Docker Engine + Compose (Linux) | recente | `docker --version` e `docker compose version` |
| Python | 3.10 a 3.12 | `python --version` |
| Java (JDK) | 17 | `java -version` |
| Editor de código (VS Code) | qualquer | abrir a pasta do projeto |

No Windows, abra o **Docker Desktop** e espere o status ficar verde ("Engine running") antes de usar qualquer comando `docker`. No Linux, confirme que o seu usuário consegue rodar `docker ps` sem `sudo`.

## Passo 1: criar a estrutura de pastas

Todo o projeto fica em uma pasta única, com o código separado da infraestrutura. Essa separação ajuda a versionar, revisar e reproduzir o ambiente.

```
seu-projeto/
├── .venv/                  (criado no Passo 2)
├── .gitignore
├── requirements.txt
├── docker/
│   ├── .env                (Passo 3, nunca vai para o Git)
│   ├── initdb/
│   └── docker-compose.yml  (Passo 4)
├── notebooks/
│   ├── 01_bronze.ipynb    (Teste do ambiente)
│   └── 02_silver.ipynb    (Transormações e primeiros Insigths)
│   └── 03_gold.ipynb      (BI)
│   └── 04_vetorial.ipynb  (Deixando tudo pronto)
└── data/                   (arquivos de entrada da atividade)
```

Crie as pastas pelo terminal, na pasta onde você guarda os seus projetos.

**Windows (PowerShell):**

```
mkdir seu-projeto
cd projeto-medalhao
mkdir docker, notebooks, data…
```

**Linux:**

```
mkdir -p seu-projeto/{docker,notebooks,data…}
cd seu-projeto
```

Abra a pasta `seu-projeto` no VS Code (*File > Open Folder*). Todos os comandos deste guia partem **da raiz dessa pasta**, salvo quando dito o contrário.

## Passo 2: ambiente virtual e dependências (Fizemos em Sala)

O ambiente virtual (`.venv`) isola as bibliotecas deste projeto das demais instalações de Python da sua máquina.

**Windows (PowerShell):**

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
```

Se o PowerShell recusar com a mensagem de execução de scripts desabilitada, rode uma vez

`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, feche e abra o terminal. No `cmd`, use `.venv\Scripts\activate.bat`. O comando `source` não existe no Windows.

**Linux:**

```
python3 -m venv .venv
source .venv/bin/activate
```

O terminal passa a mostrar `(.venv)` no início da linha. Agora crie o arquivo `requirements.txt` na raiz:

```
pyspark==3.5.3
jupyterlab
pandas>=2.2.2
pyarrow>=16.1.0
boto3>=1.34.136
psycopg2-binary>=2.9.9
fastembed          # embeddings locais (ONNX, sem GPU). Pode pular se for usar
EMBED_BACKEND=hash
```

E instale:

```
pip install -r requirements.txt
```

> ***Qualidade e governança:*** *o PySpark está com a versão* ***fixada*** *(`==3.5.3`). Versões soltas fazem o mesmo código funcionar na máquina de um colega e quebrar na de outro. Fixar versão é o primeiro passo da reprodutibilidade.*

As bibliotecas de Postgres e de embeddings ficam de fora de propósito: entram no `requirements.txt` quando montarmos essa parte juntos.

## Passo 3: variáveis de ambiente (.env) (Fizemos em Sala)

O usuário e a senha do MinIO não podem ficar escritos no `docker-compose.yml` nem nos scripts. Eles ficam em um arquivo `.env`, e o Docker Compose o lê sozinho quando está na **mesma pasta** do `docker-compose.yml`.

Crie `docker/.env`:

```
MINIO_USER=admin
MINIO_PASSWORD=admin12345
```

A senha do MinIO precisa ter **pelo menos 8 caracteres**. Para a aula, esses valores servem. Fora da aula, nunca use senha assim.

Se você quiser crie também `docker/.env.example`, um modelo **sem os valores reais, para colocar no seu github**:

```
MINIO_USER=
MINIO_PASSWORD=
```

E o `.gitignore` na raiz, para que nada sensível vá para o repositório:

```
.env
.venv/
__pycache__/
.ipynb_checkpoints/
```

> ***Qualidade e governança:*** *segredo fora do código e fora do Git é regra básica de segurança. O* `.env.example` *documenta quais variáveis existem sem expor os valores: quem clonar o projeto sabe o que precisa preencher.*

## Passo 4: docker-compose.yml com apenas o MinIO (Fizemos em Sala)

O MinIO é o nosso *object storage*: ele fala a mesma API do Amazon S3, então o código que vocês escrevem aqui funciona depois em um bucket de nuvem trocando só o endereço e as credenciais.

Crie `docker/docker-compose.yml`:

```
name: pipeline-aula

services:
  minio:
    image: cgr.dev/chainguard/minio:latest # lembre-se de verificar a imagem que deu certo
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_PASSWORD}
    ports:
      - "9000:9000"   # API S3 (usada pelo Python e pelo Spark)
      - "9001:9001"   # console web (usada por nós, no navegador)
    volumes:
      - minio_data:/data
    mem_limit: 512m

volumes:
  minio_data:
```

O que cada parte faz:

| Trecho | Função |
|---|---|
| `image` | Qual imagem baixar. Usamos a da Chainguard, porque as imagens `minio/minio` e `minio/mc` deixaram de ser distribuídas no Docker Hub e o Quay recusa o acesso. |
| `command` | Inicia o servidor guardando os dados em `/data` e abre o console na porta 9001. |
| `environment` | Recebe usuário e senha das variáveis do `.env` (`${...}`). |
| `ports` | Liga a porta do container à do seu computador (`seu-pc:container`). |

| Trecho | Função |
|---|---|
| `volumes` | Guarda os dados fora do container: eles sobrevivem a `docker compose down`. |
| `mem_limit` | Limita a memória, para não travar a máquina. |

Essa imagem não tem terminal dentro dela. Por isso o compose não tem *healthcheck* nem um serviço para criar os buckets: Essa part epode ser feita via codigo python ou de forma manual, pela interface gráfica do MinIO.

> ***Qualidade e governança:*** *uma imagem sem terminal e sem ferramentas extras tem menos superfície de ataque. Já o `:latest` é uma concessão da aula: em produção, fixe a versão (ou o digest) da imagem, pelo mesmo motivo que fixamos o PySpark.*

**Teste:** na pasta `docker`, rode `docker compose config`. Na saída, `MINIO_ROOT_USER` e `MINIO_ROOT_PASSWORD` devem aparecer preenchidos. Se estiverem em branco, o `.env` está no lugar errado ou com o nome errado (Passo 3).

## Passo 5: subir o MinIO e abrir o console

Com o Docker Desktop aberto e rodando, entre na pasta `docker` e suba o serviço:

```
cd docker
docker compose up -d
```

A primeira vez baixa a imagem e demora um pouco. Em versões antigas do Docker o comando tem hífen: `docker-compose up -d`.

Confirme que está de pé e veja o log:

```
docker compose ps
docker compose logs minio
```

Em `ps`, o serviço `minio` deve estar com estado *running*. No log, procure as linhas com o endereço da API (porta 9000) e do console (porta 9001).

Agora abra o navegador em **http://localhost:9001** e entre com o `MINIO_USER` e o `MINIO_PASSWORD` do seu `.env`. Você verá o console do MinIO, ainda sem nenhum bucket.

Comandos que vocês vão usar sempre:

| Comando | O que faz |
|---|---|
| `docker compose up -d` | Sobe os serviços em segundo plano |
| `docker compose ps` | Mostra o que está rodando |

| Comando | O que faz |
|---|---|
| `docker compose logs minio` | Mostra o log do MinIO |
| `docker compose down` | Para e remove o container. **Os dados continuam** no volume |
| `docker compose down -v` | Para e **apaga também o volume**. Os dados somem |

### Passo 6: criar os buckets `landing` e `warehouse`

Um *bucket* é o recipiente de mais alto nível do object storage. Vamos usar dois, para separar zonas de dados com finalidades diferentes:

| Bucket | Para quê | Quem escreve |
|---|---|---|
| `landing` | Zona de pouso: o arquivo exatamente como chegou da fonte | A coleta (ingestão) |
| `warehouse` | As camadas do pipeline (bronze, prata, ouro), separadas por prefixo | O Spark |

> ***Qualidade e governança:*** *manter o dado cru separado do dado tratado permite regras diferentes para cada zona: quem pode escrever, por quanto tempo guardar, o que pode ser apagado. Na prática, essas regras são políticas por bucket.*

Vou deixar aqui um codigo caso voce queira criar os buckets via codigo.

Crie `scripts/criar_buckets.py`:

```
import os
import boto3
from botocore.exceptions import ClientError
from dotenv import load_dotenv

load_dotenv("docker/.env")   # rode sempre a partir da raiz do projeto

s3 = boto3.client(
    "s3",
    endpoint_url="http://localhost:9000",
    aws_access_key_id=os.environ["MINIO_USER"],
    aws_secret_access_key=os.environ["MINIO_PASSWORD"],
    region_name="us-east-1",
)

for bucket in ["landing", "warehouse"]:
    try:
        s3.head_bucket(Bucket=bucket)
        print(f"já existe : {bucket}")
```

```
    except ClientError as e:
        if e.response["Error"]["Code"] in ("404", "NoSuchBucket"):
            s3.create_bucket(Bucket=bucket)
            print(f"criado    : {bucket}")
        else:
            raise
```

Com o `.venv` ativo, rode **a partir da raiz** do projeto:

```
python scripts/criar_buckets.py
```

A saída esperada na primeira vez é `criado : landing` e `criado : warehouse`. Rode uma segunda vez: o script diz `já existe` e não quebra. Essa propriedade, de poder repetir sem efeito colateral, chama-se **idempotência** e vai aparecer muito no pipeline.

Volte ao console (http://localhost:9001, aba *Buckets*) e confirme que os dois aparecem. Se preferir, também dá para criá-los clicando em *Create Bucket*, mas o script deixa o passo **reproduzível**.

## Passo 7: verificar o ambiente

Em vez de confiar que "deve estar tudo certo", vamos **testar**. Use o arquivo `scripts/verifica_ambiente.py`:

```
import os, shutil, sys
import boto3
from dotenv import load_dotenv
from pathlib import Path

RAIZ = Path(__file__).resolve().parent.parent      # pasta do projeto
load_dotenv(RAIZ / "docker" / ".env")
ok = True

def linha(nome, cond, dica=""):
    global ok
    ok = ok and bool(cond)
    print("OK   " if cond else "FALTA", nome, "" if cond else f"-> {dica}")

linha("Java", shutil.which("java") or os.environ.get("JAVA_HOME"),
      "instale o JDK 17 e confira o JAVA_HOME")

try:
    import pyspark
    linha(f"PySpark {pyspark.__version__}", True)
except ImportError:
    linha("PySpark", False, "pip install -r requirements.txt")

try:
    s3 = boto3.client(
        "s3",
        endpoint_url="http://localhost:9000",
```

```
        aws_access_key_id=os.environ["MINIO_USER"],
        aws_secret_access_key=os.environ["MINIO_PASSWORD"],
        region_name="us-east-1",
    )
    nomes = [b["Name"] for b in s3.list_buckets()["Buckets"]]
    linha("MinIO (buckets landing e warehouse)",
          {"landing", "warehouse"} <= set(nomes),
          "rode: python scripts/criar_buckets.py")
except Exception as e:
    linha("MinIO", False, f"cd docker && docker compose up -d ({type(e).__name__})")

try:
    import tempfile
    from pyspark.sql import SparkSession

    os.environ["PYSPARK_PYTHON"] = sys.executable
    os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

    spark = (SparkSession.builder.master("local[2]").appName("teste")
         .config("spark.driver.bindAddress", "127.0.0.1")
         .config("spark.driver.host", "127.0.0.1")
         .config("spark.ui.enabled", "false")
         .getOrCreate())
    df = spark.createDataFrame([(1, "a"), (2, "b")], ["id", "v"])
    destino = os.path.join(tempfile.gettempdir(), "teste_parquet")
    df.write.mode("overwrite").parquet(destino)          # no Windows, quebra aqui
sem o winutils
    n = spark.read.parquet(destino).count()
    spark.stop()
    linha(f"Spark {pyspark.__version__} (grava e lê Parquet)", n == 2,
          "veja a tabela de erros do Spark")
except Exception as e:
    causa = [l for l in str(e).splitlines() if l.strip()][:3]
    linha("Spark", False, " | ".join(causa))

print("\nPostgres + pgvector e Metabase: vamos montar juntos nas próximas etapas.")
print("Tudo pronto." if ok else "Resolva os itens FALTA e rode de novo.")
sys.exit(0 if ok else 1)
```

```
python scripts/verifica_ambiente.py
```

O resultado esperado:

```
OK    Java
OK    PySpark 3.5.3
OK    MinIO (buckets landing e warehouse)
FALTA Spark -> Py4JJavaError: veja a tabela de erros do Spark

Postgres + pgvector e Metabase: vamos montar juntos nas próximas etapas.
Tudo pronto.
```

> ***Qualidade e governança:*** *esse script é um teste de qualidade do* ***ambiente****. Cada* `FALTA` *traz a dica do que fazer. Em pipelines de verdade, checagens assim rodam antes de qualquer processamento, para falhar cedo e com uma mensagem clara.*

### Teste de objeto (opcional, 2 minutos)

Para ver na prática o que é um objeto e uma chave, envie um arquivo pequeno e liste por prefixo. Rode em um terminal Python (`python`) ou em uma célula de notebook, a partir da raiz:

```
import os, boto3
from dotenv import load_dotenv
load_dotenv("docker/.env")

s3 = boto3.client("s3", endpoint_url="http://localhost:9000",
                  aws_access_key_id=os.environ["MINIO_USER"],
                  aws_secret_access_key=os.environ["MINIO_PASSWORD"],
                  region_name="us-east-1")

s3.put_object(Bucket="landing", Key="teste/pasta/ola.txt", Body=b"ola, object
storage")
resp = s3.list_objects_v2(Bucket="landing", Prefix="teste/")
print([o["Key"] for o in resp["Contents"]])
```

Abra o console, entre no bucket `landing` e navegue por `teste/pasta/`. Depois apague o objeto:

```
s3.delete_object(Bucket="landing", Key="teste/pasta/ola.txt")
```

Pergunta para pensar: você criou alguma "pasta" `teste/pasta/`? Em que lugar ela existe, de fato?

## Passo 8: Configurar e testar o Spark

O Spark roda na JVM (Java). No Linux ele funciona só com Java e PySpark. No Windows ele precisa de dois arquivos extras (`winutils.exe` e `hadoop.dll`) e de uma variável de ambiente (`HADOOP_HOME`), porque a API de arquivos do Hadoop chama funções nativas do sistema.

**8.1 Java 17**

Windows (PowerShell):

```
winget install EclipseAdoptium.Temurin.17.JDK
```

Feche e abra o terminal e confira com `java -version`. Depois defina o `JAVA_HOME`, ajustando o número da versão para o que foi instalado:

```
[Environment]::SetEnvironmentVariable("JAVA_HOME","C:\Program Files\Eclipse
Adoptium\jdk-17.0.13.11-hotspot","User")
```

Linux (Ubuntu/Debian ou WSL2):

```
sudo apt update && sudo apt install -y openjdk-17-jdk-headless
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc
echo 'export PATH=$JAVA_HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

```
java -version
```

**8.2 Hadoop (somente Windows)**

No Linux **pule para o 8.3**.

1. Crie a pasta dos binários:

```
mkdir C:\hadoop\bin
```

2. No repositório https://github.com/cdarlint/winutils, abra a pasta hadoop-3.3.6/bin e baixe winutils.exe e hadoop.dll. Coloque os dois em C:\hadoop\bin.
**3.** Os dois arquivos (`winutils.exe` e `hadoop.dll`) estão na pasta `winutils` do repositório da turma. Copie-os para `C:\hadoop\bin`

*(Origem dos arquivos: github.com/cdarlint/winutils, pasta `hadoop-3.3.6/bin`. Só baixe de lá se a pasta da turma não tiver os arquivos.)*
4. Confira se os arquivos estão lá (os dois comandos devem responder True):

```
Test-Path C:\hadoop\bin\winutils.exe
Test-Path C:\hadoop\bin\hadoop.dll
```

5. Defina a variável e adicione a pasta ao PATH:
```
[Environment]::SetEnvironmentVariable("HADOOP_HOME","C:\hadoop","User")
$p = [Environment]::GetEnvironmentVariable("Path","User")
[Environment]::SetEnvironmentVariable("Path", "$p;C:\hadoop\bin", "User")
```

6. **Feche o VS Code por completo e abra de novo.** O Jupyter herda as variáveis do VS Code, e o VS Code só lê as novas quando é iniciado de novo. Reiniciar apenas o kernel não basta.(*As vezes reiniciar o pc também resolve =/*)
Se o `hadoop.dll` der erro de DLL, instale o *Visual C++ Redistributable 2015-2022*.

**8.3 PySpark(Se precisar)**

Com o .venv ativo, o PySpark já veio no requirements.txt (pyspark==3.5.3). Se precisar instalar à parte:

```
bash
pip install pyspark==3.5.3
```

**8.4 Testar o Spark**

```
python scripts/verifica_ambiente.py
```

O resultado esperado:

```
OK Java
OK PySpark 3.5.3
OK Spark 3.5.3 (grava e lê Parquet)
```

```
OK MinIO (buckets landing e warehouse)

Postgres + pgvector e Metabase: vamos montar juntos nas próximas etapas.
Tudo pronto.
```

**8.5 Contorno rápido caso ainda não tenha reconhecido no Windows (sem reabrir o VS Code)**

Se o erro continuar depois de definir o `HADOOP_HOME`, rode esta célula **antes** de qualquer código Spark, em um kernel recém-reiniciado:

```
import os
os.environ["HADOOP_HOME"] = r"C:\hadoop"
os.environ["PATH"] = r"C:\hadoop\bin;" + os.environ["PATH"]
```

A variável precisa existir **antes** de o Spark iniciar a JVM. Se o Spark já tentou subir naquela sessão, reinicie o kernel antes

## Erros comuns e como resolver

| Mensagem ou sintoma | Causa provável | O que fazer |
|---|---|---|
| `cannot connect to the Docker daemon` / `error during connect` | Docker Desktop fechado | Abrir o Docker Desktop e esperar o status "Engine running" |
| `pull access denied` ou `unauthorized` ao baixar a imagem | Imagem antiga (`minio/minio`, `minio/mc` ou Quay) no compose | Usar `cgr.dev/chainguard/minio:latest`, como no Passo 4 |
| `variable is not set. Defaulting to a blank string` | `.env` fora da pasta `docker` ou com nome errado | Deixar o `.env` ao lado do `docker-compose.yml` |
| `port is already allocated` (9000 ou 9001) | Outro programa usa a porta | Trocar o mapeamento (ex.: `"9100:9000"`) e usar a nova porta no `endpoint_url` |
| `EndpointConnectionError` / `Could not connect to the endpoint URL` | MinIO parado | `docker compose ps` e `docker compose logs minio` |
| `InvalidAccessKeyId` / `SignatureDoesNotMatch` | Usuário ou senha do `.env` diferentes dos do container | Corrigir o `.env`, depois `docker compose down` e `docker compose up -d` |
| `KeyError: 'MINIO_USER'` | Script rodado fora da raiz do projeto, então o `docker/.env` não foi lido | Voltar à raiz e rodar `python scripts/...` |
| PowerShell: "a execução de | Política de execução do | `Set-ExecutionPolicy -Scope` |

| Mensagem ou sintoma | Causa provável | O que fazer |
|---|---|---|
| scripts foi desabilitada" | Windows | `CurrentUser RemoteSigned` e reabrir o terminal |
| Container `minio` reinicia ou sai logo após subir | Erro de permissão no volume ou variável faltando | `docker compose logs minio`. Se citar permissão em `/data`, rode `docker compose down -v` e suba de novo |
| `FALTA MinIO (buckets landing e warehouse)` com o MinIO rodando | Buckets ainda não criados | `python scripts/criar_buckets.py` (Passo 6) |

*Se nada disso resolver, copie a mensagem de erro **inteira** e o resultado de `docker compose ps` e leve para a turma.*

**8.6 Erros do Spark**

| Mensagem | Causa | Correção |
|---|---|---|
| `JAVA_HOME is not set` / `Java gateway process exited` | Java ou `JAVA_HOME` ausente | Passo 8.1 e reabrir o terminal |
| `HADOOP_HOME and hadoop.home.dir are unset` | Falta o 8.2 (Windows) ou o VS Code não foi reaberto | Definir `HADOOP_HOME`, reabrir o VS Code ou usar o contorno do 8.5 |
| `UnsatisfiedLinkError ... NativeIO$Windows` | `hadoop.dll` ausente ou incompatível | Copiar o `hadoop.dll` para `C:\hadoop\bin` |
| `Python worker failed to connect back` | Python errado ou versão 3.13 | Usar Python 3.10 a 3.12 e definir `PYSPARK_PYTHON` com o python do `.venv` |
| `Unsupported class file major version` | Java mais novo que 17 | Instalar o Java 17 e apontar o `JAVA_HOME` para ele |

**Qualidade e governança:** o teste grava e lê um arquivo de verdade, em vez de só checar se o pyspark importa. Verificar o comportamento, e não a presença do programa, pega o erro antes do pipeline.

***Plano B:** quem não conseguir deixar o Spark funcionando segue no Google Colab com o mesmo código, e resolve o ambiente local depois.*

## Checklist final e próximos passos

Marque o que você já conferiu:

- ☐ `docker compose ps` mostra o serviço `minio` rodando
- ☐ O console abre em http://localhost:9001 com o usuário e a senha do `.env`
- ☐ Os buckets `landing` e `warehouse` aparecem no console
- ☐ `python scripts/verifica_ambiente.py` mostra `OK` em Java, PySpark e MinIO
- ☐ O `.env` está no `.gitignore` e o `.env.example` está sem valores
- ☐ Eu sei explicar com minhas palavras o que é um bucket, um objeto e uma chave

**Depois disso, montamos juntos:** o Postgres com a extensão pgvector, o Metabase e as tabelas da arquitetura para o banco de dados vetorial. Por isso o seu `docker-compose.yml` hoje tem só o MinIO: ele vai ganhar novos serviços, um de cada vez, a cada etapa.